

Prédiction des cartons jaunes

Projet de Machine Learning — apprentissage supervisé & non supervisé

FIFA World Cup 2026 — Player Performance Dataset

Kiritheepan Robinsan
ENSEA · Barcelona Summer Course 2026
Variable cible : **yellow_cards** (0 = pas de carton, 1 = carton)

Jeu de données	54 600 lignes × 75 colonnes — 1 248 joueurs — 0 valeur manquante
Modèles supervisés	Arbre de décision · Forêt aléatoire · Gradient Boosting
Interprétabilité	Importance native & permutation · SHAP (summary + waterfall)
Non supervisé	PCA · ICA · t-SNE · KMeans · DBSCAN
Métrique principale	F1-score (classes déséquilibrées ~90 / 10)
Reproductibilité	Pipeline re-exécuté indépendamment : tous les chiffres confirmés

0. Contexte, données et préparation

L'objectif est de prédire, pour un joueur lors d'un match, s'il recevra un **carton jaune**. La cible **yellow_cards** est fortement déséquilibrée, et c'est précisément ce qui structure toute la démarche.

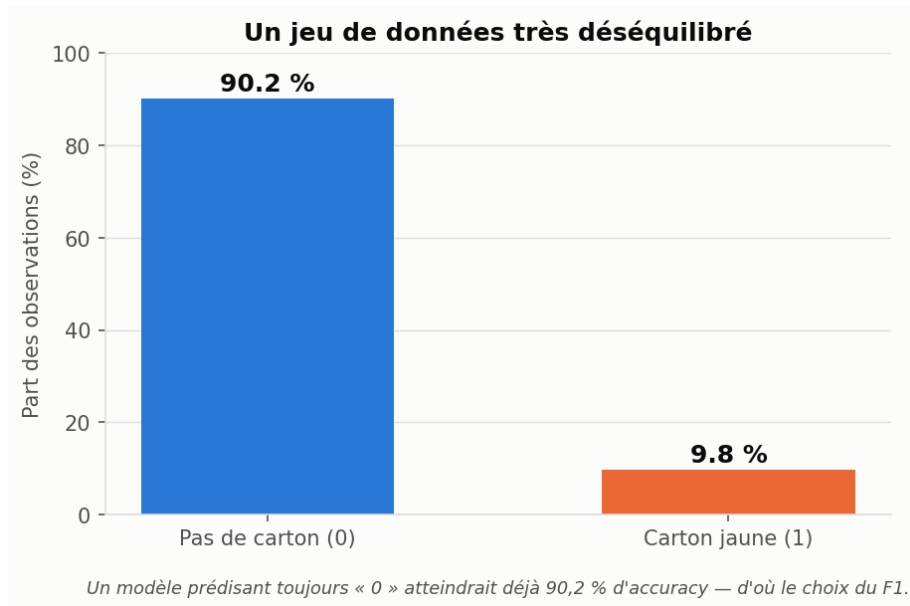


Figure 1 — Répartition de la variable cible. Le déséquilibre 90/10 rend l'accuracy trompeuse : c'est la raison du choix du F1-score comme métrique de référence.

Qualité des données

Le jeu de données est propre : **aucune valeur manquante** et aucune valeur physiquement impossible (vérifié en triant les colonnes par leur minimum et en filtrant les valeurs négatives).

Corrélations avec la cible

Première alerte sur la difficulté du problème : **aucune variable n'est corrélée linéairement** à la cible.

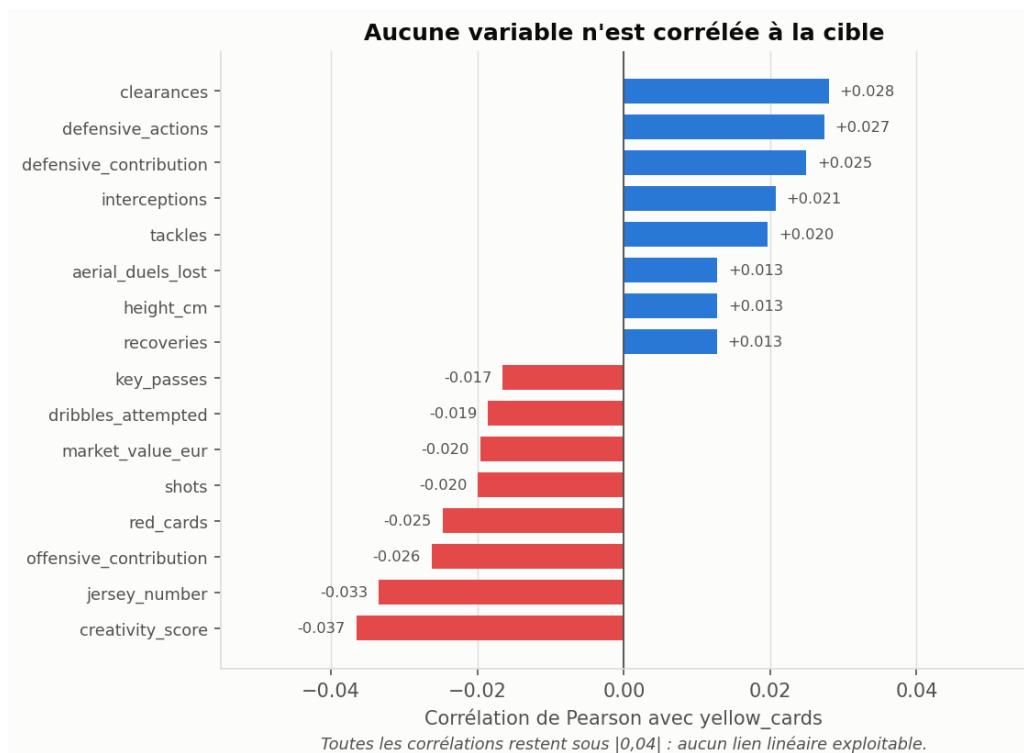


Figure 2 — Les 8 corrélations les plus positives et les 8 plus négatives. Toutes restent sous $|0,04|$. On note toutefois une cohérence footballistique : les variables défensives (dégagements, tacles, interceptions) sont les seules du côté positif.

Ce résultat ne condamne pas le projet : la corrélation ne mesure que les liens **linéaires simples**. Il justifie au contraire l'emploi de modèles à base d'arbres, capables de capter des **combinaisons** de variables.

Encodage et exclusions

Les identifiants (`player_id`, `player_name`, `match_id`, `match_date`, `stadium`, `city`) et les variables à trop forte cardinalité (`nationality` : 48 valeurs, `club_name` : 122, `team`, `opponent_team`) sont **exclus** : aucune valeur prédictive et risque de sur-apprentissage. Les variables catégorielles utiles (`position`, `preferred_foot`, `tournament_stage`, `match_result`) sont encodées en one-hot avec `drop_first=True`.

Séparation train / test par joueur

Chaque joueur apparaît environ 44 fois (une ligne par match). Un découpage ligne par ligne placerait le **même joueur des deux côtés** : le modèle le reconnaîtrait au lieu d'apprendre à généraliser — une **fuite de données** qui gonflerait artificiellement les scores. On sépare donc les **1 248 joueurs** en 80 % / 20 %, ce qui donne **43 726** lignes d'entraînement et **10 874** de test, avec **zéro joueur en commun** (vérifié).

1. Apprentissage supervisé

Trois modèles à base d'arbres sont entraînés puis réglés par GridSearchCV (scoring="f1", validation croisée), et évalués sur le même jeu de test.

Le déséquilibre des classes : un piège concret

Sans correction, la forêt aléatoire prédit **systématiquement 0** et obtient un **F1 de 0** : noyée par 90 % de « pas de carton », elle choisit la facilité. Deux mécanismes corrigent cela en pondérant ~9× plus fortement les erreurs sur la classe rare : `class_weight="balanced"` (forêt) et un `sample_weight` équilibré (le boosting n'ayant pas de `class_weight` natif).

Meilleurs hyperparamètres retenus

Résultat de la recherche par GridSearchCV pour chaque modèle :

Modèle	Meilleurs hyperparamètres	F1 (validation croisée)
Arbre de décision	<code>max_depth = None · min_samples_leaf = 1</code>	0,113
Random Forest	<code>n_estimators = 100 · max_depth = 5 min_samples_leaf = 1 · class_weight = balanced</code>	0,185
Gradient Boosting	<code>n_estimators = 100 · learning_rate = 0,05 max_depth = 3 · sample_weight équilibré</code>	0,479

Tableau 1 — Meilleurs hyperparamètres trouvés par GridSearchCV (scoring = F1).

Le F1 de validation croisée du Gradient Boosting (0,479) est nettement supérieur à son F1 de test (0,175). L'explication tient au découpage interne de GridSearchCV : il sépare les plis **ligne par ligne**, si bien qu'un même joueur peut se retrouver des deux côtés pendant la validation interne — ce qui gonfle le score. Le jeu de test final, séparé **par joueur**, est plus sévère et donne la mesure honnête de la performance.

Métrique	Arbre de décision	Random Forest	Gradient Boosting
Accuracy	0,816	0,661	0,643
Précision	0,100	0,116	0,111
Rappel	0,125	0,407	0,412
F1-score	0,111	0,181	0,175
ROC-AUC	0,506	0,546	0,543

Tableau 2 — Comparaison sur le jeu de test (10 874 lignes, joueurs jamais vus à l'entraînement).

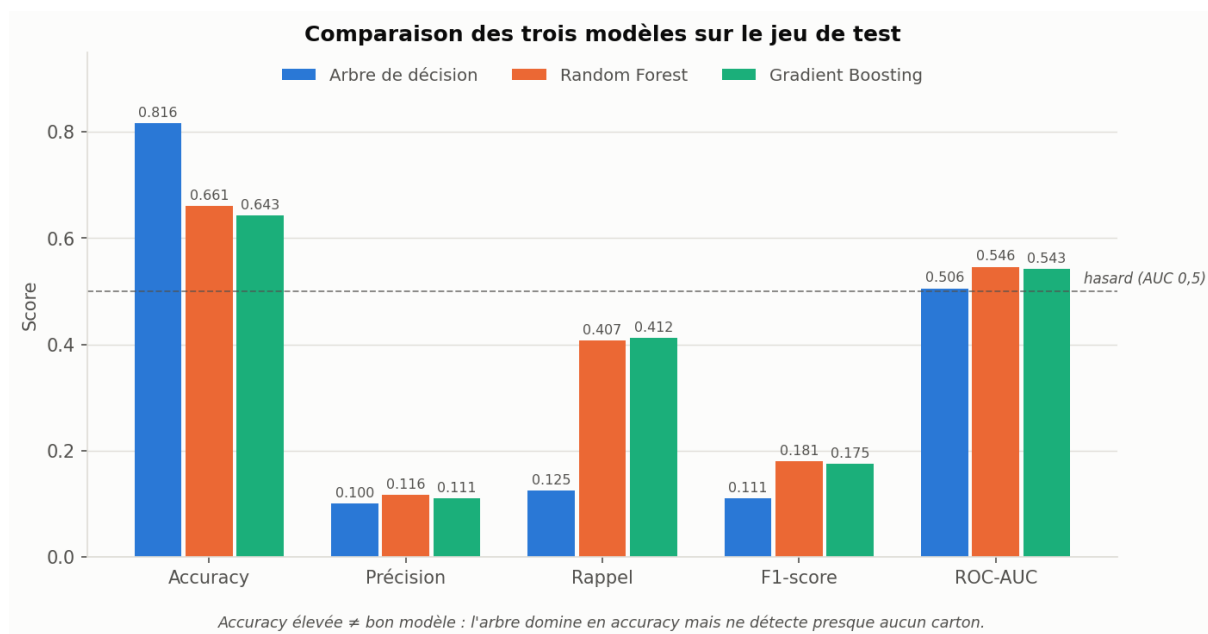


Figure 3 — Les mêmes résultats en visuel. L'arbre domine en accuracy tout en étant le plus mauvais sur le rappel : l'illustration parfaite du piège de l'accuracy.

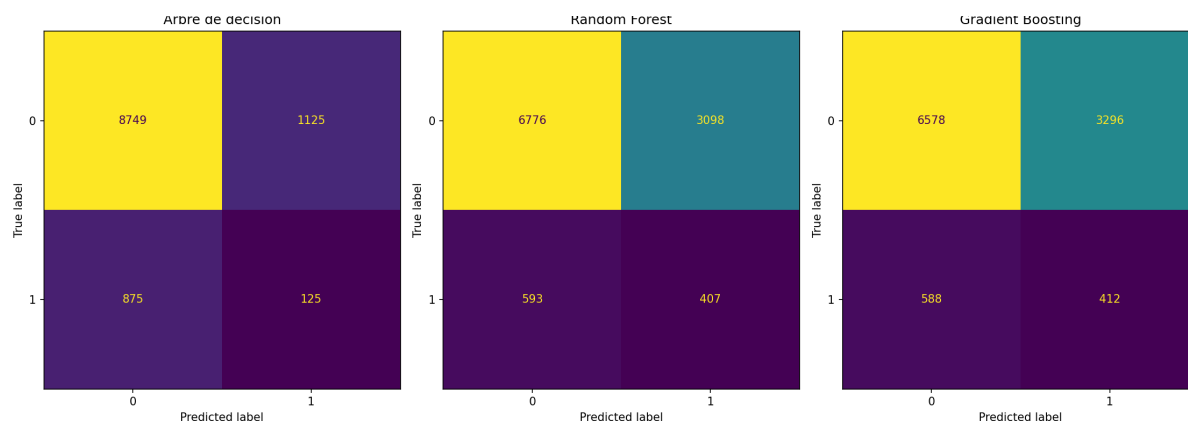


Figure 4 — Matrices de confusion. Diagonale = bonnes prédictions ; hors-diagonale = erreurs (haut-droite = faux positifs, bas-gauche = faux négatifs).

Lecture des résultats

L'arbre seul obtient la meilleure accuracy (0,816) mais ne détecte que **125 cartons sur 1 000** (rappel 12,5 %). Pire : un modèle trivial prédisant toujours « pas de carton » atteindrait ~90,2 % d'accuracy — **l'arbre fait donc moins bien qu'un modèle qui n'apprend rien**. Le rééquilibrage fait bondir le rappel de la forêt et du boosting à ~40 %, au prix d'une accuracy plus faible. Sur le critère F1, **Random Forest l'emporte de peu** (0,181 contre 0,175).

Faux positifs et faux négatifs, dans le contexte du football

Un **faux positif** revient à signaler à tort qu'un joueur risque un carton ; un **faux négatif** à ne pas anticiper un carton réel. Dans ce contexte, le faux positif paraît le plus coûteux : on ne veut pas pénaliser ou stigmatiser un joueur injustement.

Or la Random Forest produit **3 098 faux positifs contre 593 faux négatifs** — elle se trompe donc plus de **5 fois plus souvent dans le sens jugé le plus grave**. Sur les 3 505 alertes « carton » qu'elle émet, seules 407 sont justes (précision 11,6 %) : environ **9 alertes sur 10 sont infondées**. Optimiser le F1 ne correspond donc pas à la priorité métier réelle ; une optimisation orientée **précision**, ou un relèvement du seuil de décision, serait préférable pour cet usage.

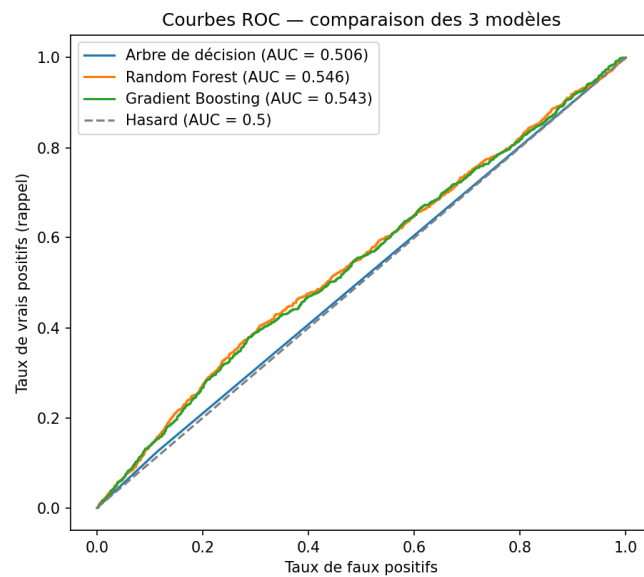


Figure 5 — Courbes ROC. Les trois AUC (0,506 / 0,546 / 0,543) frôlent la diagonale du hasard (0,5).

La courbe ROC apporte l'éclairage décisif : elle teste **tous les seuils de décision**, pas seulement 50 %. Le gain de F1 obtenu par rééquilibrage traduisait surtout un **déplacement du seuil** (un modèle plus « alarmiste »), et non une réelle capacité à séparer les deux classes — capacité qui reste, elle, à peine meilleure que le hasard.

2. Importance des variables & SHAP

L'analyse porte sur le modèle retenu (Random Forest, meilleur F1).

Un piège d'interprétation : le biais de cardinalité

L'importance native (Gini) place `jersey_number` — le numéro de maillot — en tête, ce qui n'a **aucun sens footballistique**. C'est un biais documenté : cette mesure favorise les variables comportant beaucoup de valeurs distinctes, car elles offrent mécaniquement plus d'occasions de découpage aux arbres.

L'**importance par permutation** tranche : elle mélange aléatoirement une colonne à la fois et mesure la chute réelle du F1. Le verdict est sans appel — `jersey_number` passe de 0,128 à 0,008, et toutes les importances s'effondrent d'un facteur ~ 16 .

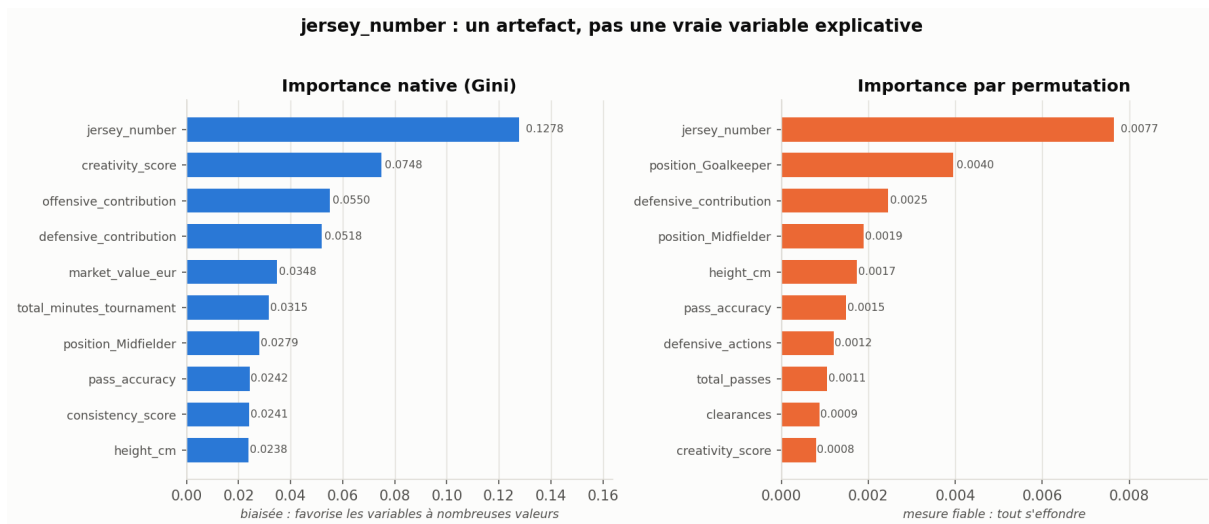


Figure 6 — Importance native (biaisée) vs importance par permutation (fiable). Noter les échelles des deux axes : 0–0,16 à gauche, 0–0,008 à droite. Aucune variable n'a d'effet réellement marquant.

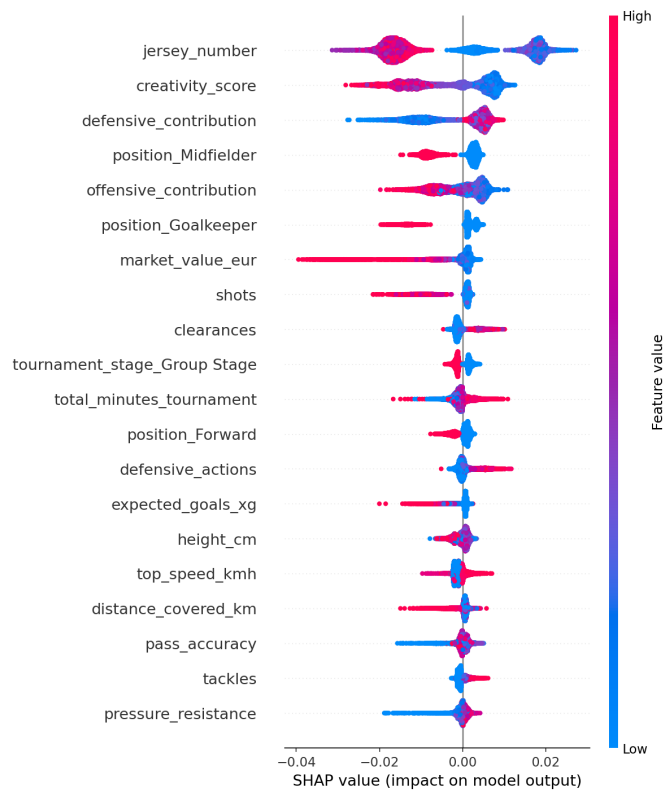


Figure 7 — SHAP summary plot. Chaque point est un joueur ; rouge = valeur haute de la variable, bleu = valeur basse ; à droite = pousse vers « carton ». *defensive_contribution* montre la relation la plus cohérente (rouge à droite : plus d'engagement défensif, plus de risque). L'échelle (−0,04 à +0,03) confirme un signal ténu.

Figure 8 — SHAP waterfall pour un joueur. On part de la prédiction moyenne (0,5) et on arrive à 0,443 : bleu = pousse vers « pas de carton », rouge = vers « carton ». Ici, un *creativity_score* très élevé (99) réduit le risque ; *total_minutes_tournament* pousse en sens inverse, mais de façon négligeable.

3. Apprentissage non supervisé

La cible n'est **pas** utilisée pour construire les groupes : elle sert uniquement à interpréter les résultats *a posteriori*. Les variables sont d'abord standardisées — contrairement aux arbres, PCA et KMeans sont très sensibles à l'échelle (sans quoi `market_value_eur`, en millions, écraserait `age`, en dizaines).

Trois techniques de réduction de dimension comparées

Une seule méthode ne suffit pas à conclure : on compare **PCA** (linéaire, cherche la variance maximale), **ICA** (linéaire, cherche des composantes statistiquement *indépendantes*) et **t-SNE** (non linéaire, préserve le *voisinage local*).

t-SNE ayant une complexité quadratique, il est appliqué à un **échantillon de 5 000 points**, précédé d'une PCA à 30 composantes — pratique standard qui accélère le calcul et débruite les données.

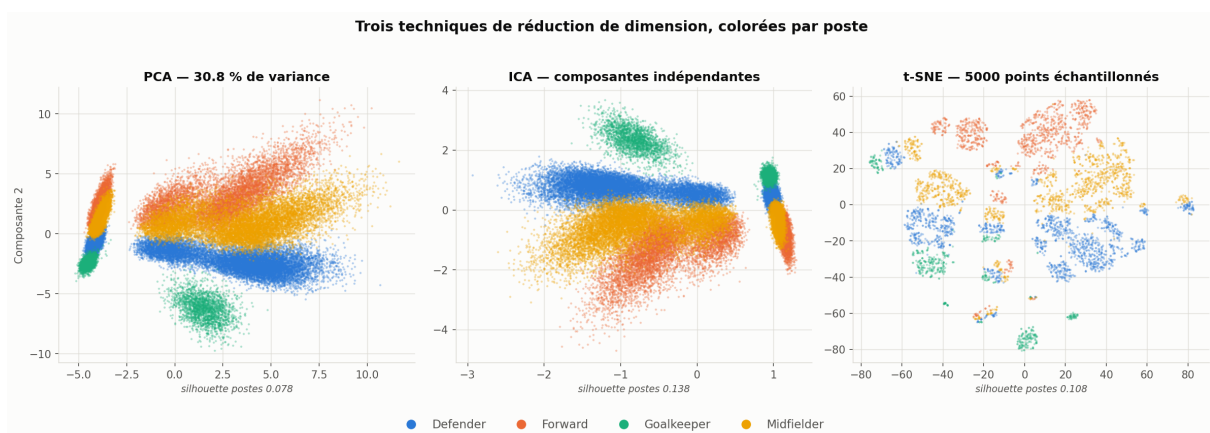


Figure 9 — Les trois projections, colorées par poste (usage post-hoc des labels : ils ne servent qu'à interpréter). PCA isole les gardiens mais mélange les autres postes ; ICA fait apparaître des bandes horizontales nettes par rôle ; t-SNE éclate les données en de nombreux micro-groupes.

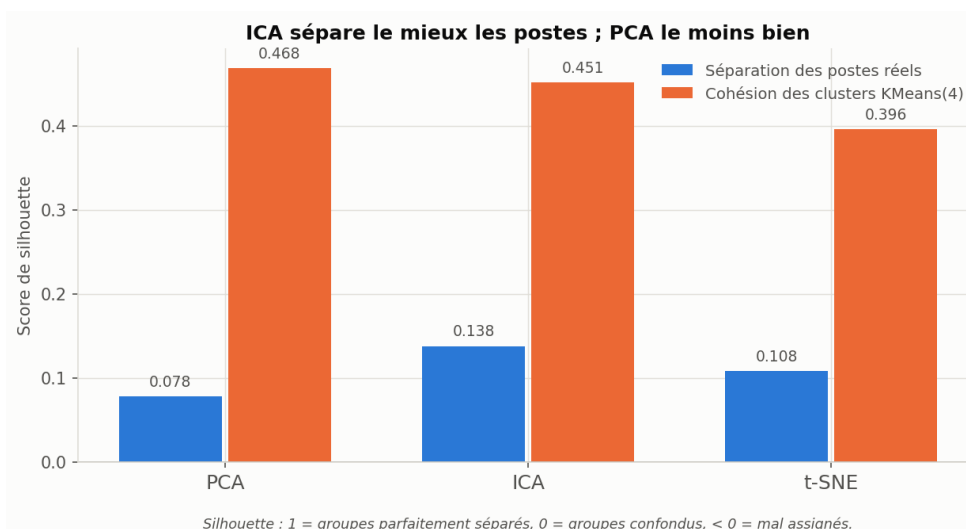


Figure 10 — Comparaison chiffrée par score de silhouette (1 = groupes parfaitement séparés).

Technique	Séparation des postes	Cohésion KMeans(4)	Temps de calcul
PCA	0,078	0,468	< 1 s (54 600 lignes)
ICA	0,138	0,451	< 1 s (54 600 lignes)
t-SNE	0,108	0,396	19 s (5 000 lignes)

Tableau 3 — Comparaison quantitative des trois techniques (meilleurs scores surlignés).

Ce que révèle la comparaison. ICA sépare le mieux les postes réels (0,138 contre 0,078 pour PCA) : chercher des composantes indépendantes s'avère plus pertinent ici que chercher la variance maximale, qui ne coïncide pas avec ce qui distingue les rôles. t-SNE, malgré son rendu visuel spectaculaire, obtient un score intermédiaire (0,108) — ses micro-groupes traduisent des similarités locales très fines plutôt que la structure des postes. PCA conserve en revanche la meilleure cohésion de clustering (0,468) et reste de loin la plus rapide sur le jeu complet : c'est donc elle qu'on retient pour la suite.

Point commun aux trois : aucune ne fait apparaître de structure liée aux **cartons jaunes**. Toutes révèlent le profil de jeu et le poste — ce qui confirme, par une quatrième voie indépendante, le constat central du projet.

PCA en détail

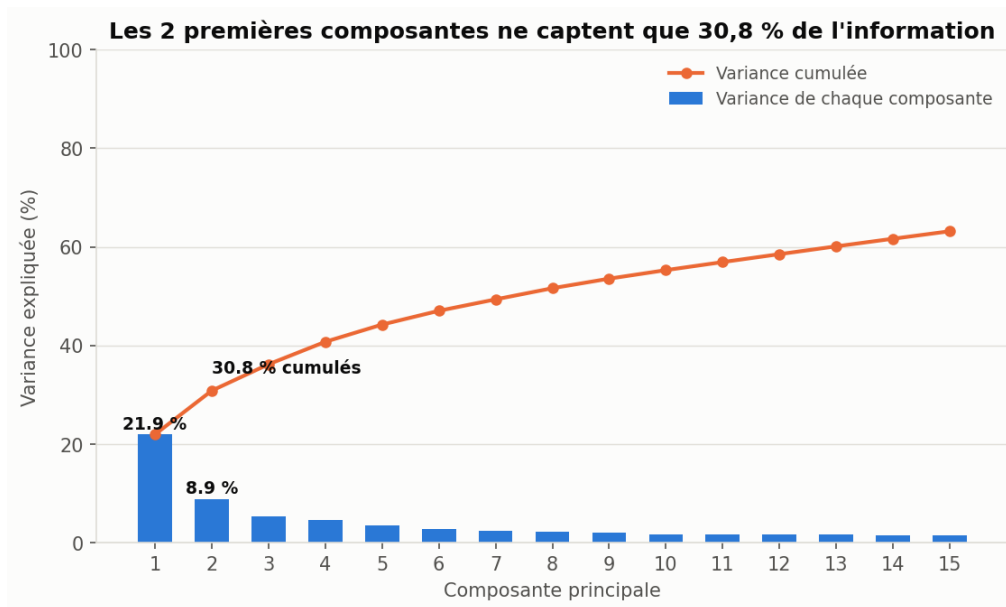


Figure 11 — Variance expliquée par composante. Les deux premières ne captent que 30,8 % de l'information : la projection 2D est utile pour visualiser, mais en perd près de 70 %.

PCA — ce que révèle la projection

Malgré cette perte, une **structure nette** apparaît — et elle s'explique par le **poste** des joueurs : les gardiens, dont le profil statistique est radicalement différent (peu de tacles, beaucoup d'arrêts), forment un groupe bien séparé.

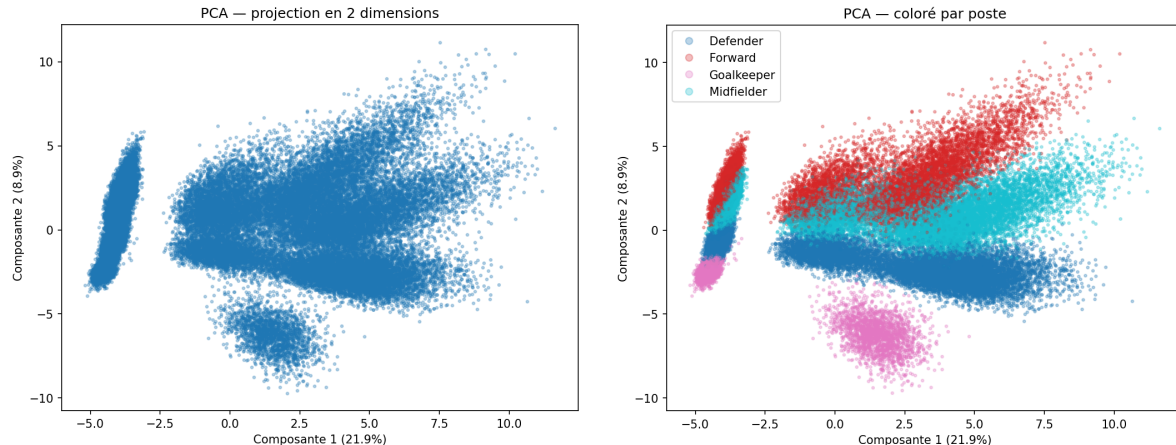


Figure 12 — Projection PCA brute (gauche) et colorée par poste (droite). Les gardiens (rose) se détachent nettement ; défenseurs, milieux et attaquants s'organisent en bandes.

Clustering — KMeans face à DBSCAN

KMeans ($k = 4$, comme les 4 postes) donne un **silhouette score de 0,135**, faible : les groupes se chevauchent. Il ne retrouve pas les postes réels et fusionne notamment l'amas des gardiens avec un autre groupe — ce qui s'explique : KMeans cherche des groupes **convexes de taille comparable**.

DBSCAN, qui raisonne par **densité** sans imposer de forme, se comporte différemment. Appliqué sur les 72 dimensions il classe tout en « bruit » (les distances explosent en haute dimension — la « malédiction de la dimension ») ; appliqué sur la projection PCA avec $\text{eps}=0.8$, il isole **2 clusters nets** (31 531 et 23 042 lignes) et seulement 27 points de bruit.

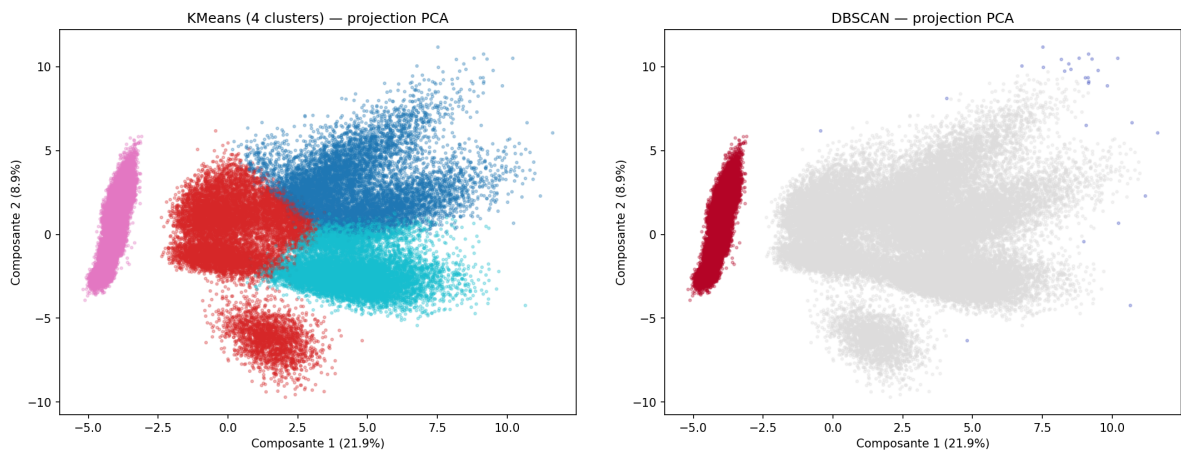


Figure 13 — KMeans (gauche, 4 clusters) et DBSCAN (droite, 2 clusters + bruit) projetés sur les axes PCA.

Ce que disent les clusters — et ce qu'ils ne disent pas

	Cluster 0	Cluster 1
Taille	31 531 lignes	23 042 lignes
Part de gardiens	6,7 %	18,2 %
Taux de cartons jaunes	9,9 %	9,6 %

Tableau 4 — Le cluster 1 est presque 3× plus riche en gardiens, mais le taux de cartons est quasi identique dans les deux (ligne surlignée).

Conclusion de cette partie : la structure naturelle des données reflète le **profil physique et le poste** des joueurs, mais elle **n'a aucun lien avec les cartons jaunes**.

Conclusion générale

Quatre approches indépendantes convergent vers le même constat :

- **Corrélation** — aucune variable au-dessus de $|0,04|$;
- **Modèles supervisés** — AUC de 0,506 à 0,546, à peine mieux que le hasard ;
- **Importance par permutation & SHAP** — effets individuels minuscules ;
- **Clustering** — la structure trouvée reflète le poste, pas les cartons.

Les cartons jaunes ne sont pas prédictibles à partir de ces variables. La convergence de quatre méthodes de natures différentes rend cette conclusion solide : ce n'est pas l'échec d'un modèle mal réglé, mais une propriété du jeu de données.

L'explication la plus plausible est que le comportement disciplinaire dépend de facteurs **absents du dataset** : la décision de l'arbitre (et sa sévérité propre), le contexte instantané du match, la tension d'une action isolée. Aucune statistique agrégée de performance ne capture ces éléments.

Ce que la démarche démontre

- le **piège de l'accuracy** sur données déséquilibrées — l'arbre « bat » les autres en accuracy tout en étant inutilisable ;
- le **rééquilibrage des classes** par `class_weight` et `sample_weight`, et son effet mesurable (F1 de 0 à 0,18) ;
- l'**arbitrage précision / rappel**, et le fait qu'une métrique d'optimisation puisse diverger de la priorité métier ;
- le **biais de cardinalité** des importances natives, diagnostiqué puis corrigé par permutation ;
- la prévention d'une **fuite de données** par un découpage train/test par joueur ;
- la différence de comportement entre **KMeans** (groupes convexes) et **DBSCAN** (densité), et la malédiction de la dimension.

Reproductibilité

L'intégralité du pipeline a été **ré-exécutée indépendamment** à partir du jeu de données brut : répartition des classes, découpage train/test (et l'absence de joueur commun), matrices de confusion et les cinq métriques des trois modèles, importances, variance PCA, silhouette, tailles des clusters DBSCAN et taux de cartons par cluster. **Tous les résultats sont reproduits à l'identique.**

Un faible pouvoir prédictif, établi rigoureusement et interprété lucidement, reste un résultat de data science parfaitement valable — et souvent plus instructif qu'un score flatteur mal contrôlé.